BUỔI CHIA SẺ KIẾN THỨC DEV · UNITY 6

Chuyện gì xảy ra sau mỗi dòng code Unity?

GetComponent, Destroy, Update, coroutine, prefab… Unity thực sự làm gì bên dưới, và nên viết thế nào cho đúng

[Tên người trình bày] · [Ngày trình bày]

4 phần · 6 demo trong Unity · github.com/vtcong2299/DEMO_UNITY

MỞ ĐẦU

Hai buổi trước và hôm nay

Buổi Debug

Khi lỗi đã xảy ra: dùng công cụ nào để **tìm ra** nó.

Buổi Claude

Để AI viết code, mình chuẩn bị đầu vào và **kiểm tra** đầu ra.

Hôm nay

Hiểu Unity làm gì bên dưới, để **không viết ra** lỗi và review được code AI viết.

MỞ ĐẦU

Cách đọc bài

3 mức độ

CƠ BẢN Đã biết, nói lướt

TRUNG BÌNH Đã dùng nhưng hay hiểu sai

NÂNG CAO Nằm sâu trong engine, có demo

DEMO Slide cam: chuyển sang Unity

TỔNG KẾT Cuối mỗi phần: nên viết thế nào

Mỗi mục đi 4 bước

1. Thường viết
2. Thật ra Unity làm gì
3. Hậu quả
4. Nên viết

MỞ ĐẦU · TỰ TEST LẠI SAU BUỔI HỌP

Mọi demo đều ở trên GitHub

1. git clone hoặc Download ZIP

2. Unity Hub → Add → mở bằng **Unity 6.4**

3. Đợi import, scene tự được dựng nếu cần

4. Mở **Assets/_Talk/Scenes/01_FakeNull**, Game view 16:9, bấm Play

5. Làm theo bảng bước trong tài liệu; chuyển scene ở đáy màn hình

PROJECT DEMO

`github.com/vtcong2299/DEMO_UNITY`

TÀI LIỆU

`TalkDocs_UNITY/SauMoiDongCodeUnity.docx`

Màn hình demo: **trái** là các bước, **phải** là bảng Kết quả (chấm **xanh** = đúng, chấm **đỏ** = bẫy).

QUIZ · GIƠ TAY TRƯỚC KHI XEM ĐÁP ÁN

Đúng hay sai?

#	Câu hỏi	Mức
1	enabled = false thì coroutine của script đó dừng	Cơ bản
2	Time.timeScale = 0 thì Update ngừng chạy	Cơ bản
3	Ngay sau Destroy(obj), obj == null là true	Trung bình
4	FixedUpdate luôn chạy đúng 1 lần mỗi frame	Trung bình
5	target?.Attack() an toàn với object đã Destroy	Nâng cao
6	async void tự dừng khi object bị Destroy	Nâng cao

PHẦN 1 / 4

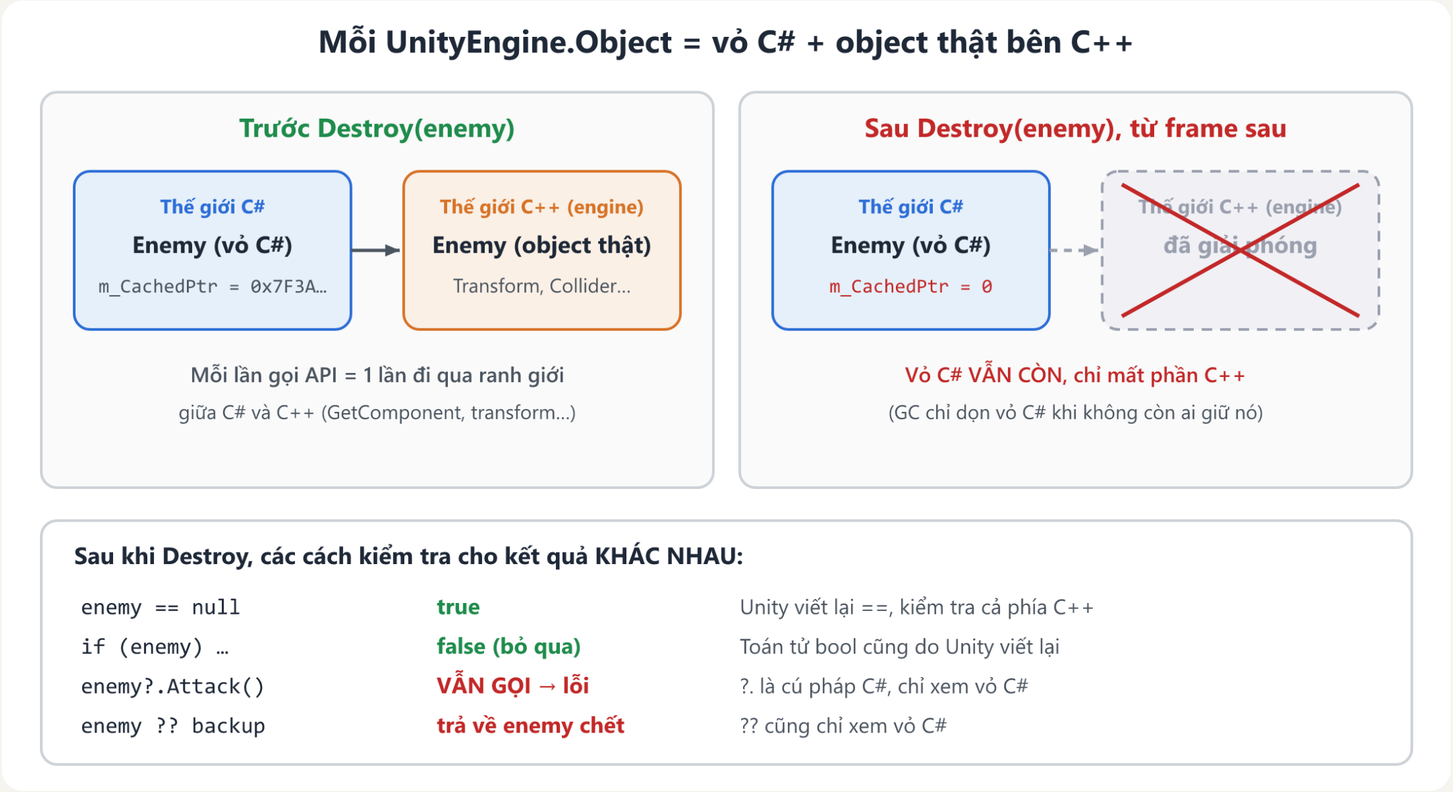
Hai thế giới: C# và C++

Mỗi object Unity là một vỏ C# trỏ sang object thật bên C++.

Demo: 01_FakeNull · 02_NativeCost

PHẦN 1 · HAI THẾ GIỚI TRUNG BÌNH

Một object, hai nửa



Vỏ C#: thứ mình cầm trong code, giữ con trỏ m_CachedPtr.

Object C++: object thật trong engine.

Mỗi lần gọi API là một lần **qua ranh giới**.

Gọi 1 lần thì rẻ, gọi mỗi frame thì không

THƯỜNG VIẾT

```
void Update()
{
  GetComponent<Rigidbody>()
      .AddForce(Vector3.up);
  transform.position = new Vector3(
    transform.position.x + 1,
    transform.position.y,
    transform.position.z);
}
```

NÊN VIẾT

```
Rigidbody _rb;
void Awake() ⇒
  _rb = GetComponent<Rigidbody>();
void Update()
{
  var p = transform.position; // đọc 1 lần
  p.x += 1;
  transform.position = p;    // ghi 1 lần
}
```

Đoạn bên trái: mỗi frame tìm component 1 lần, đọc **position** 3 lần, ghi 1 lần.

Cùng cơ chế, gặp ở khắp nơi

Ở đâu	Mỗi lần gọi, Unity làm gì	Nên dùng
mesh.vertices	Copy toàn bộ mảng đỉnh sang mảng C# mới	Đọc 1 lần, GetVertices(list)
renderer.materials	Mảng mới + nhân bản mọi material	sharedMaterials
Input.touches	Tạo mảng Touch[] mới	touchCount + GetTouch(i)
GetComponentsInChildren	Duyệt cả cây con, tạo mảng mới	Overload nhận List<T>
FindObjectOfType, Find	Duyệt toàn bộ scene	Kéo thả reference, tìm 1 lần
Camera.main	Trước 2020.2: tìm theo tag. Nay: đã cache	Bản mới gọi thoải mái

Lời khuyên tối ưu có **hạn sử dụng**: hiểu nguyên lý mới biết lời khuyên nào đã cũ.

Fake null: vì sao ?. không an toàn

```
Destroy(enemy.gameObject);
// ... từ frame sau
enemy == null      // true
if (enemy)         // false, bỏ qua
enemy is null      // FALSE
enemy?.Attack();   // VẪN GỌI, lỗi
enemy ?? backup    // enemy đã chết
```

Unity **viết lại** == và phép ép sang bool: kiểm tra cả phía C++.

?. ?? is null là cú pháp C#, không viết lại được: chỉ nhìn vỏ C#.

Destroy chỉ có hiệu lực ở **cuối frame**.

Với object Unity: chỉ dùng **if (obj != null)** hoặc **if (obj)**.

▶ DEMO 1 / 6 · CHUYỂN SANG UNITY

01_FakeNull

Câu hỏi: sau Destroy, 5 cách kiểm tra null có cho cùng kết quả?

#	Bấm	Kết quả đúng
1	Destroy Enemy rồi kiểm tra bằng 5 cách	== null: True · if (enemy): False · is null: **False** · ?.Attack(): **vẫn gọi, lỗi** · ?? backup: **enemy đã chết**
2	GetComponent trên object không có Rigidbody	== null: True · is null: **False** (Editor) · TryGetComponent: False
3	Tạo lại Enemy	Khối đỏ xuất hiện lại, làm lại bước 1

Assets/_Talk/Scenes/01_FakeNull.unity · github.com/vtcong2299/DEMO_UNITY

Hàm Update rỗng vẫn tốn

Unity tìm hàm **Update** bằng reflection lúc load script, rồi **mỗi frame gọi từ C++ sang C#** từng hàm một.

Thân hàm rỗng vẫn tốn chi phí của lần gọi qua ranh giới.

1.000 đồng xu có Update rỗng = **1.000 lần gọi vô ích** mỗi frame.

Xoá Update rỗng. Nhiều object giống nhau: **1 manager** gọi Tick().

NÊN VIẾT

```
class CoinManager : MonoBehaviour
{
  readonly List<Coin> _coins = new();

  void Update()  // 1 lần C++ → C#
  {
    float dt = Time.deltaTime;
    for (int i = 0; i < _coins.Count; i++)
      _coins[i].Tick(dt);  // C# → C#
  }
}
```

▶ DEMO 2 / 6 · CHUYỂN SANG UNITY

02_NativeCost

Câu hỏi: gọi API Unity tốn bao nhiêu? Đo bằng số.

#	Bấm	Kết quả đúng
1	GetComponent mỗi lần vs lưu sẵn	Lưu sẵn nhanh hơn nhiều lần
2	Đọc position 3 lần vs 1 lần	Đọc 1 lần nhanh hơn rõ rệt
3	tag == vs CompareTag	tag == tạo **vài MB** rác, CompareTag **0 KB**
4	GameObject.Find vs reference (+5.000 object)	Find chậm, thêm object càng chậm
5	Không Update → Update rỗng → Manager	Update rỗng tốn nhất, Manager rẻ hơn

Assets/_Talk/Scenes/02_NativeCost.unity · github.com/vtcong2299/DEMO_UNITY

TỔNG KẾT PHẦN 1 · HAI THẾ GIỚI C# VÀ C++

Nên viết thế nào

Tình huống	Nên viết	Tránh
Dùng component nhiều lần	Lấy 1 lần trong Awake, lưu field; hoặc kéo thả	GetComponent trong Update
Kiểm tra tag	CompareTag("Enemy")	tag == "Enemy"
Đọc/ghi transform	Đọc 1 lần vào biến, ghi 1 lần	Đọc position nhiều lần trong 1 dòng
Tìm object khác	Kéo thả; hoặc FindAnyObjectByType 1 lần	Find trong Update
Object Unity còn sống?	if (obj) hoặc obj != null	?. ?? is null
Component có thể không có	TryGetComponent(out T c)	GetComponent rồi quên kiểm tra
Nhiều object cùng cập nhật	Xoá Update rỗng; 1 manager gọi Tick()	Mỗi object một Update rỗng

PHẦN 2 / 4

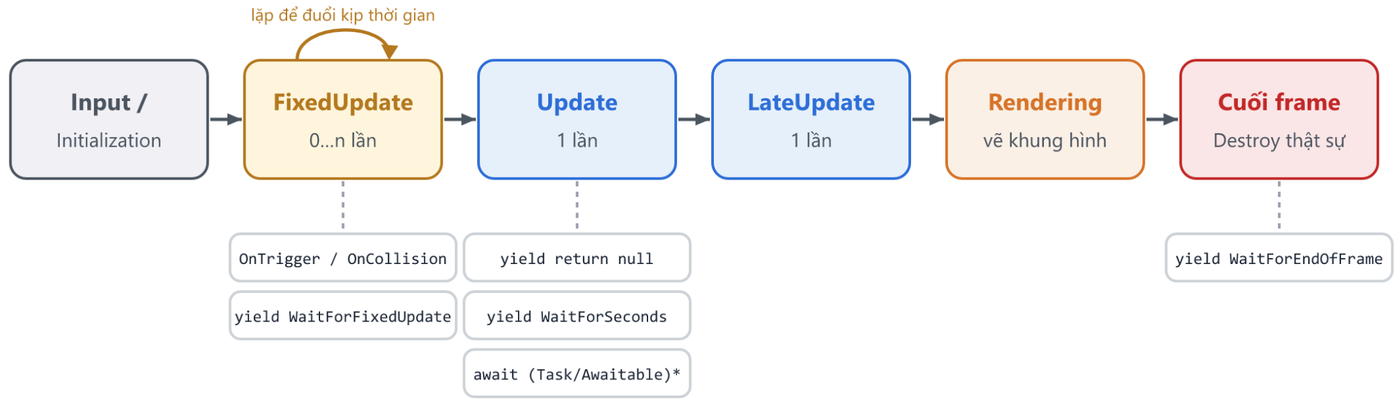
Một frame trong Unity chạy những gì

Biết code của mình chạy ở bước nào thì hiểu được phần lớn lỗi về thứ tự và thời gian.

Demo: 03_PlayerLoop · 04_Lifetime

PHẦN 2 · PLAYER LOOP TRUNG BÌNH

Thứ tự cố định trong mỗi frame



Camera bám nhân vật bị giật? Nhân vật đi theo **vật lý** (50 lần/giây), camera theo **frame**: bật Interpolate hoặc bám trong LateUpdate.

Vòng đời: những chi tiết hay quên

Hàm	Chạy lại?	Chi tiết hay quên
Awake	Không, 1 lần	Chạy cả khi component đang tắt
OnEnable	Có, mỗi lần bật lại	Đăng ký event ở đây
Start	Không	Chỉ chạy khi component đang bật
OnDisable	Có, mỗi lần tắt	Huỷ đăng ký event ở đây
OnDestroy	Không	Chỉ khi object từng được bật

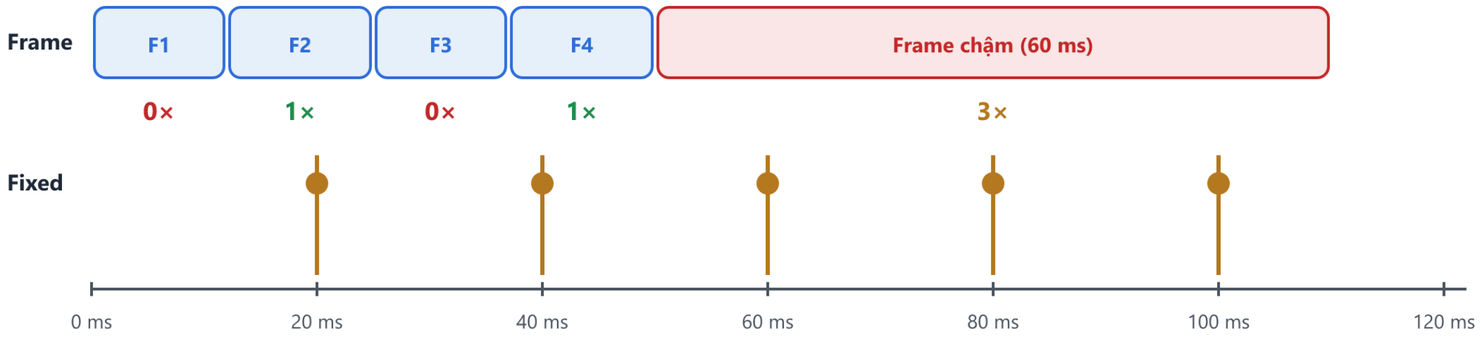
Đăng ký ở OnEnable, huỷ ở OnDestroy: panel bật/tắt 3 lần thì handler chạy 3 lần.

Thứ tự Awake giữa các script **không được đảm bảo**: “máy tôi chạy, máy bạn lỗi”.

PHẦN 2 · PLAYER LOOP TRUNG BÌNH

FixedUpdate: 0 lần, 1 lần hay nhiều lần

FixedUpdate không gắn với frame: 0 lần, 1 lần hay nhiều lần



fixedDeltaTime = 0.02 s → cứ mỗi 20 ms "thời gian game" có 1 bước vật lý. Frame nhanh: có thể 0 lần. Frame chậm: chạy bù nhiều lần.

GetKeyDown trong FixedUpdate: frame có 0 lần FixedUpdate thì lần bấm bị mất.

Đọc input ở **Update**, lưu cờ, xử lý vật lý ở **FixedUpdate**.

Máy chậm thì chạy bù. **Maximum Allowed Timestep** chặn vòng xoáy.

PHẦN 2 · PLAYER LOOP TRUNG BÌNH

timeScale = 0: cái gì dừng, cái gì không

Thứ	Khi timeScale = 0	Muốn chạy lúc pause
Update, LateUpdate	Vẫn chạy, deltaTime = 0	unscaledDeltaTime
FixedUpdate, vật lý	Dừng hẳn	-
WaitForSeconds	Đứng yên mãi	WaitForSecondsRealtime
Animator	Đứng yên	Update Mode = Unscaled Time
ParticleSystem	Đứng yên	Main → Use Unscaled Time
DOTween	Đứng yên	.SetUpdate(true)
AudioSource	**Vẫn phát**	Tự pause âm thanh

Lỗi “menu pause bị đơ”: popup dùng WaitForSeconds hoặc Animator mặc định.

▶ DEMO 3 / 6 · CHUYỂN SANG UNITY

03_PlayerLoop

Câu hỏi: FixedUpdate có chạy 1 lần mỗi frame? timeScale = 0 thì cái gì dừng?

#	Bấm	Kết quả đúng
1	Target FPS: Không giới hạn	Rất nhiều frame có **0 lần** FixedUpdate
2	Làm 1 frame chậm 100 ms	Frame sau chạy bù khoảng **5 lần**
3	Làm 1 frame chậm 1000 ms	Khoảng **17 lần**, không phải 50
4	Tự bấm nút Nhảy 50 lần	Đọc trong FixedUpdate **mất** lần bấm; đọc ở Update **100%**
5	Pause game (timeScale = 0)	Update vẫn chạy, FixedUpdate = 0; khối đỏ dừng, khối xanh xoay

Assets/_Talk/Scenes/03_PlayerLoop.unity · github.com/vtcong2299/DEMO_UNITY

PHẦN 2 · COROUTINE TRUNG BÌNH

Coroutine thuộc về ai gọi StartCoroutine

```
class A : MonoBehaviour
{
  public B b;
  void Start() ⇒
    StartCoroutine(b.Scroll());  // A là chủ
}
class B : MonoBehaviour
{
  public IEnumerator Scroll()
  {  yield return new WaitForSeconds(2f);
     Debug.Log("After 2 seconds");  }
}
b.StartCoroutine(b.Scroll());  // B là chủ
```

Khi...	A là chủ	B là chủ
B bị tắt	**Vẫn chạy**	Dừng
B bị Destroy	**Vẫn chạy**, dễ lỗi	Dừng
A bị tắt	Dừng	**Vẫn chạy**
b.StopAllCoroutines()	Không dừng	Dừng

Nên: B có hàm **PlayScroll()** tự chạy coroutine của mình; A chỉ gọi **b.PlayScroll()**.

Coroutine và async sống tới khi nào?

Coroutine

enabled = false → **vẫn chạy**

SetActive(false) → dừng, bật lại **không tự chạy**

Thuộc về object **gọi** StartCoroutine

```
async void Start()
{  await Task.Delay(3000);
   transform.position = Vector3.zero; }
// object chết, thoát Play: vẫn chạy
```

```
async Awaitable Start()  // Unity 6
{  await Awaitable.WaitForSecondsAsync(
      3f, destroyCancellationToken);
   transform.position = Vector3.zero; }
```

Unity 2022.3: Task.Delay(ms, destroyCancellationToken) hoặc UniTask.

▶ DEMO 4 / 6 · CHUYỂN SANG UNITY

04_Lifetime

Câu hỏi: tắt script, tắt GameObject, Destroy, thoát Play: cái gì còn chạy?

#	Bấm	Kết quả đúng
1	Tắt script (enabled = false)	Coroutine của Worker **vẫn chạy**
3 / 3b	StartCoroutine(worker....) / worker.StartCoroutine(worker....)	Cả hai cùng chạy
4	Tắt GameObject Worker	Coroutine Worker và **3b dừng**; **3 vẫn chạy** (Demo là chủ)
5	Bật GameObject lại	Coroutine Worker **không tự chạy lại**
6	Destroy Worker	Task **vẫn chạy**; Awaitable + token **dừng**
8	Stop, mở Console	"[Task] Vẫn chạy sau khi đã thoát Play mode"

Assets/_Talk/Scenes/04_Lifetime.unity · github.com/vtcong2299/DEMO_UNITY

TỔNG KẾT PHẦN 2 · MỘT FRAME CHẠY NHỮNG GÌ

Nên viết thế nào

Tình huống	Nên viết	Tránh
Đăng ký sự kiện	OnEnable đăng ký, OnDisable huỷ	OnEnable đăng ký, OnDestroy huỷ
Khởi tạo phụ thuộc nhau	Awake lo bản thân, Start lấy object khác; chuỗi Initialize()	Dựa vào thứ tự Awake
Coroutine điều khiển object khác	Để object đó tự chạy: b.PlayScroll()	StartCoroutine(b.Scroll()) từ A
Coroutine lặp	Cache WaitForSeconds	new WaitForSeconds mỗi vòng
Menu pause	unscaledDeltaTime, WaitForSecondsRealtime	WaitForSeconds cho UI lúc pause
Input điều khiển vật lý	Đọc ở Update, xử lý ở FixedUpdate	GetKeyDown trong FixedUpdate
Chờ bằng async	Awaitable + destroyCancellationToken	async void + Task.Delay trần

PHẦN 3 / 4

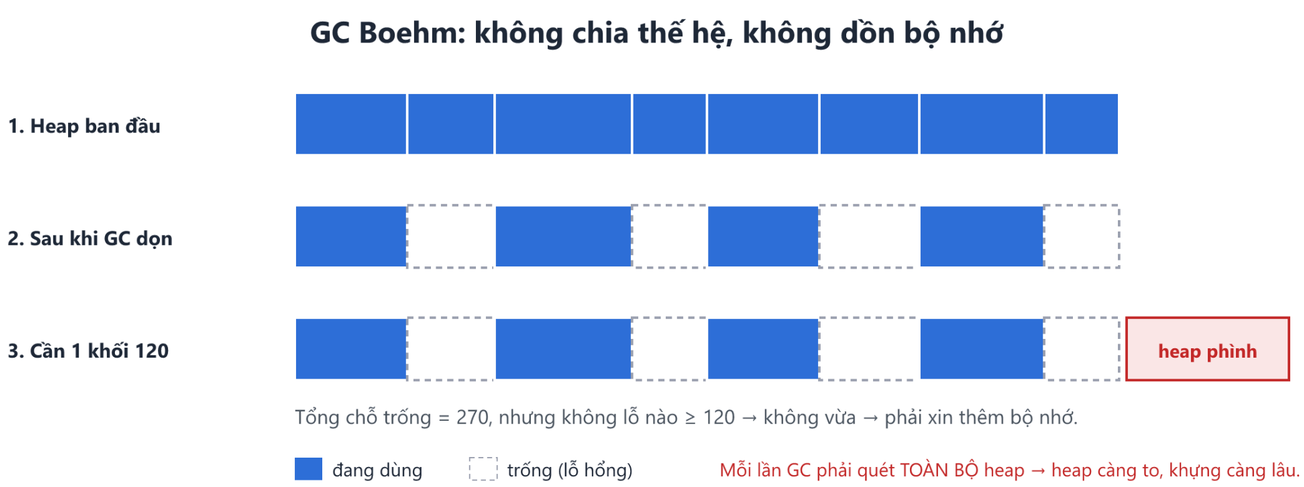
Bộ nhớ và Garbage Collector

FPS trung bình 60 vẫn có thể giật: thủ phạm thường là GC.

Demo: 06_GarbageCollector

PHẦN 3 · BỘ NHỚ NÂNG CAO

GC của Unity dọn rác thế nào



Không chia thế hệ

Mỗi lần dọn quét toàn bộ heap.

Không dồn bộ nhớ

Phân mảnh, heap phình, hiếm khi co.

16,6 ms mỗi frame

Một lần GC trên điện thoại là đủ khựng.

PHẦN 3 · BỘ NHỚ NÂNG CAO

GC chạy khi nào? Chạy nhiều có lag hơn không?

Khi nào chạy

Không theo lịch. Khi một lần **new** không còn đủ chỗ trống trong heap (“heap đầy”), hoặc khi gọi GC.Collect().

Dọn xong vẫn thiếu → heap lớn lên.

Bao lâu một lần

Tốc độ tạo rác ÷ chỗ trống. 10 MB trống:

10 KB/frame → khoảng 17 giây một lần.

100 KB/frame → khoảng 1,7 giây một lần.

Có lag hơn không

Mỗi lần GC kéo dài một frame. Càng nhiều lần → càng nhiều frame bị khựng.

Incremental: chia ra nhiều frame (≈ 3 ms/frame), ít khựng nhưng vẫn tốn CPU.

Chữa gốc: **giảm rác mỗi frame** → GC thưa đi. Dọn chủ động ở màn loading.

PHẦN 3 · BỘ NHỚ TRUNG BÌNH

Rác ngầm: nhìn code không thấy

Đoạn code	Tạo rác?	Thay bằng
"Score: " + score mỗi frame	**Có**	Chỉ gán khi đổi; TMP SetText("{0}", v)
foreach trên List<T>	Không	-
foreach qua IEnumerable<T>	**Có**	Dùng đúng kiểu List<T>, hoặc for
Lambda dùng biến bên ngoài	**Có**	Vòng lặp for thường
LINQ Where().ToList()	**Có**	for + list dùng lại
new WaitForSeconds() trong vòng lặp	**Có**	Tạo 1 lần, dùng lại
GetComponents<T>(), RaycastAll	**Có**	Bản nhận List / NonAlloc
gameObject.tag, enum.ToString()	**Có**	CompareTag; bảng tra string
StartCoroutine mỗi frame	**Có**	1 coroutine sống lâu

▶ DEMO 5 / 6 · CHUYỂN SANG UNITY

06_GarbageCollector

Câu hỏi: đoạn code nào tạo rác, mỗi lần gọi bao nhiêu byte? GC dày lên ra sao?

#	Bấm	Kết quả đúng
1	Ô đỏ: ghép chuỗi, $"...", LINQ, lambda dùng biến ngoài, boxing...	Ghép chuỗi ~80, lambda ~150, **LINQ ~480 B/lần**
2	Ô xanh: foreach List, lambda không biến ngoài, CompareTag, GetComponents(list)	**≈ 0 B/lần**
3	Chạy LINQ 20 giây, rồi foreach List 20 giây	LINQ: GC trong 10 giây nhiều hơn; frame có GC **~30 ms** so với ~12 ms
4	Cấp phát 64 MB, rồi Bỏ đi + GC.Collect()	Used tăng rồi giảm; Reserved **giữ nguyên**

Assets/_Talk/Scenes/06_GarbageCollector.unity · github.com/vtcong2299/DEMO_UNITY

TỔNG KẾT PHẦN 3 · BỘ NHỚ VÀ GC

Nên viết thế nào

Tình huống	Nên viết	Tránh
Hiển thị số trên UI	Chỉ cập nhật khi đổi; SetText("{0}", x)	Ghép chuỗi gán text mỗi frame
Duyệt danh sách mỗi frame	for, hoặc foreach trên đúng kiểu List<T>	LINQ; foreach qua IEnumerable
Điều kiện lọc	Vòng lặp for với biến cục bộ	Lambda dùng biến bên ngoài
Raycast, lấy nhiều component	RaycastNonAlloc, GetComponents(list)	RaycastAll, GetComponents<T>()
Giảm khựng do GC	Incremental GC; GC.Collect ở màn loading	GC.Collect giữa gameplay
Texture, mesh, material tạo lúc chạy	Destroy khi không dùng nữa	Để GC lo (GC không quản lý native)

PHẦN 4 / 4

Rendering: những dòng code “vô hại”

Vài dòng rất bình thường có thể tăng draw call, rò rỉ bộ nhớ, hoặc làm cả Canvas dựng lại.

Demo: 07_Rendering

PHẦN 4 · RENDERING CƠ BẢN

Đọc số liệu: draw call, batch, SetPass

Số liệu	Là gì	Đọc thế nào
SetPass Calls	Một lần đổi shader pass / trạng thái render	**Đắt nhất phía CPU**: nhìn số này trước
Batches	Nhóm lần vẽ được chuẩn bị và gửi cùng nhau	Gộp được càng nhiều càng ít
Draw Calls	Lệnh "vẽ mesh này" CPU gửi GPU	Nhiều chưa chắc chậm nếu được SRP Batcher gộp
Triangles / Vertices	Số tam giác / đỉnh GPU xử lý	Liên quan độ chi tiết mesh

100 khối, 1 đèn có bóng → khoảng **200 draw call**: mỗi khối vẽ một lần cho **bóng**, một lần cho **màu**.

Muốn biết vì sao không gộp được: **Frame Debugger** ghi lý do từng lần vẽ.

PHẦN 4 · RENDERING CƠ BẢN

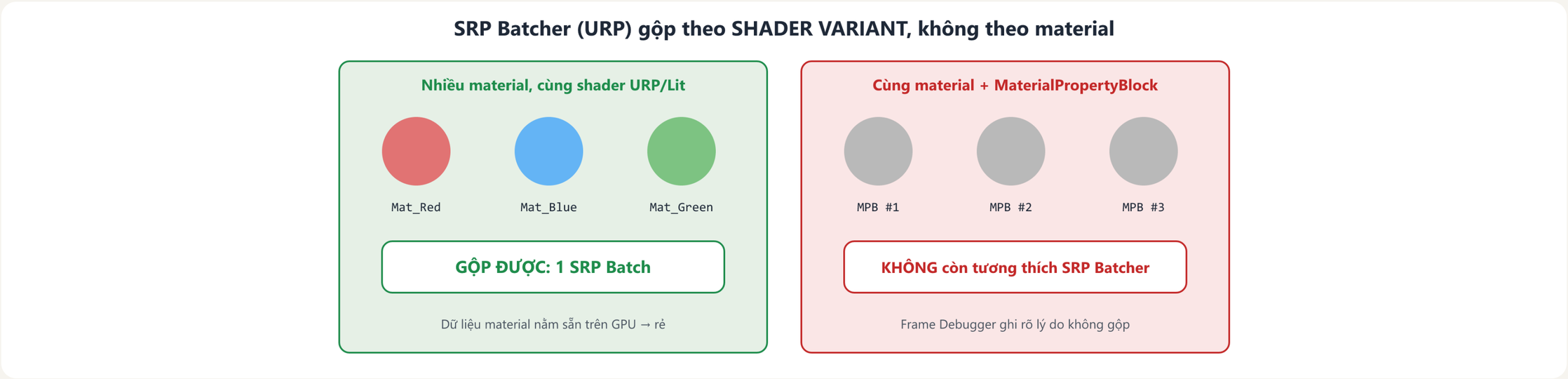
Lấy ra là nhân bản

Gọi	Unity làm gì	Rẻ hơn
renderer.material	Lần đầu: tạo **bản sao** material, không tự xoá	sharedMaterial, hoặc tự Destroy
meshFilter.mesh	Lần đầu: tạo bản sao mesh	sharedMesh
renderer.materials	Nhân bản tất cả, mảng mới	sharedMaterials
TMP fontMaterial	Bản sao material font	fontSharedMaterial
SetTrigger("Jump")	Băm chuỗi thành số mỗi lần	Animator.StringToHash

SpriteRenderer.color và UI Graphic.color là **màu đỉnh**: không đụng material, không phá batching.

PHẦN 4 · RENDERING NÂNG CAO

Lời khuyên cũ có ngoại lệ



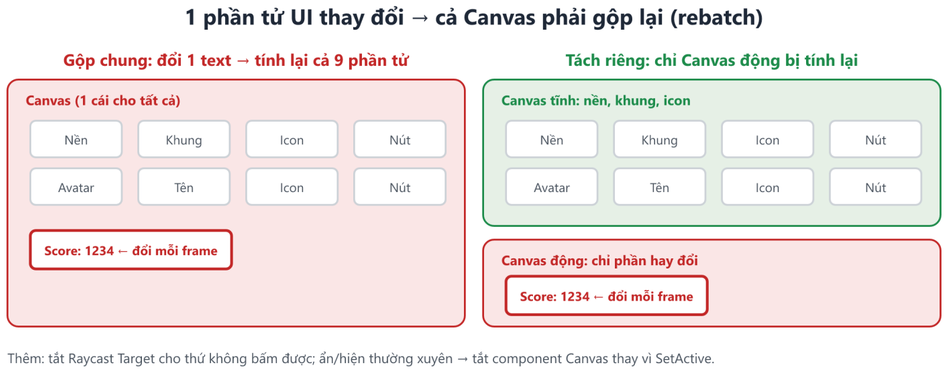
Built-in RP: MaterialPropertyBlock là cách rẻ để đổi màu từng object.

URP: gộp theo **shader variant**, MPB làm object **rơi khỏi** SRP Batcher.

Đo bằng **Frame Debugger** trên đúng pipeline trước khi tin.

PHẦN 4 · RENDERING NÂNG CAO

Đổi 1 con số, dựng lại cả Canvas



Thêm: tắt Raycast Target cho thứ không bấm được; ẩn/hiện thường xuyên → tắt component Canvas thay vì SetActive.

Cái gì làm Canvas “bẩn”

Bất kỳ phần tử con nào đổi **text, màu, sprite, fillAmount, vị trí, scale** hoặc bật/tắt. Gán lại **cùng giá trị** thì Unity bỏ qua.

Dựng lại lần vẽ

Gom lại mesh của **mọi** phần tử trong Canvas đó, không chỉ phần tử vừa đổi. Profiler: **Canvas.BuildBatch**.

Dựng lại layout

Kích thước đổi trong **LayoutGroup / ContentSizeFitter**: tính lại cả nhóm, lan lên tới nhóm ngoài cùng. Profiler: **Canvas.SendWillRenderCanvases**.

PHẦN 4 · RENDERING NÂNG CAO

Ví dụ: cái gì làm Canvas dựng lại mỗi frame

Thường viết	Unity phải làm lại	Nên viết
timerText.text = $"{t:F2}" mỗi frame, chung Canvas HUD	**Cả HUD** dựng lại mỗi frame + rác string	Canvas con cho timer; chỉ gán khi số giây đổi
Icon loading xoay, nút “Chơi” nhịp scale bằng Animator/tween lặp	Ghi RectTransform mỗi frame: Canvas chứa nó luôn bẩn, kể cả màn hình “đứng yên”	Canvas con riêng; dừng tween khi panel ẩn
Thanh máu hồi dần, thanh tải chạy liên tục	fillAmount đổi mỗi frame	Canvas con cho các thanh động
List 100 dòng trong VerticalLayoutGroup + ContentSizeFitter, đổi text 1 dòng	**Tính lại layout cả danh sách**, rồi dựng lại Canvas	Cố định chiều cao dòng; ScrollRect + pool
Popup mở/đóng liên tục bằng SetActive	Mỗi lần bật: OnEnable mọi phần tử, layout, toàn bộ mesh	canvas.enabled = false/true (giữ mesh)
30 thanh máu World Space chung 1 Canvas	Một con trúng đòn: cả 30 thanh dựng lại	Canvas nhỏ cho từng thanh, hoặc SpriteRenderer

```
int sec = Mathf.CeilToInt(remain);  if (sec ≠ _lastSec) { _lastSec = sec; timerText.SetText("{0}", sec); }
```

▶ DEMO 6 / 6 · CHUYỂN SANG UNITY

07_Rendering

Mỗi bước tự chụp “Draw (SRP Batcher) · SetPass” vào bảng Kết quả. ~70 là của giao diện demo: so phần chênh lệch.

#	Bấm	Kết quả đúng
1	sharedMaterial	Mốc: SRP Batcher 200 draw (bóng + màu); SetPass ~75; material = 0
2	renderer.material (bản sao)	Material = **100**; SetPass **vẫn ~75**
5	Ở chế độ 2: tắt SRP Batcher	SetPass **~175** (+100: mỗi material một lần); bật lại thì giảm
3	Tắt dọn bản sao, tạo lại nhiều lần	200, 300, 400… = **rò rỉ**
4	MaterialPropertyBlock	Material = 0, SRP Batcher = 0, SetPass **~275**: bình thường trong URP

Assets/_Talk/Scenes/07_Rendering.unity · github.com/vtcong2299/DEMO_UNITY

TỔNG KẾT PHẦN 4 · RENDERING

Nên viết thế nào

Tình huống	Nên viết	Tránh
Đọc số liệu rendering	Nhìn SetPass, Batches trước; Frame Debugger tìm lý do	Chỉ nhìn Draw Calls
Đổi màu một object lâu dài	material 1 lần, lưu lại, Destroy khi xong	Gọi renderer.material tuỳ tiện
Đổi màu cả nhóm	sharedMaterial	Đổi từng renderer.material
Màu sprite / UI	SpriteRenderer.color, Graphic.color	Material riêng cho sprite/UI
Tham số Animator / Shader	StringToHash, PropertyToID lưu sẵn	Truyền chuỗi mỗi lần
Nhiều object khác màu (URP)	Nhiều material cùng shader	Mặc định MaterialPropertyBlock
UI đổi liên tục / ẩn hiện nhiều	Tách Canvas động; canvas.enabled	1 Canvas cho cả màn hình; SetActive liên tục

KẾT · ĐÁP ÁN QUIZ

Cả 6 câu đều Sai

#	Câu	Vì sao
1	enabled = false dừng coroutine	Chỉ tắt GameObject hoặc Destroy mới dừng
2	timeScale = 0 dừng Update	Update vẫn chạy, deltaTime = 0
3	Ngay sau Destroy, obj == null	Destroy có hiệu lực ở cuối frame
4	FixedUpdate 1 lần mỗi frame	0 lần hoặc nhiều lần
5	?. an toàn với object đã Destroy	?. không dùng == của Unity
6	async void tự dừng khi Destroy	Task không biết vòng đời object

KẾT · CHEAT SHEET

10 thói quen nên đổi

Thường viết	Nên viết
GetComponent trong Update	Lấy 1 lần trong Awake, lưu field
other.tag == "Enemy"	other.CompareTag("Enemy")
obj?.Method(), obj ?? x với object Unity	if (obj != null) hoặc if (obj)
Đăng ký OnEnable, huỷ OnDestroy	Đăng ký OnEnable, huỷ OnDisable
GetKeyDown trong FixedUpdate	Đọc ở Update, xử lý ở FixedUpdate
async void + Task.Delay	destroyCancellationToken, Awaitable
LINQ, lambda dùng biến ngoài trong Update	Vòng for; lambda không dùng biến ngoài
"Score: " + score mỗi frame	Chỉ gán khi đổi; SetText("{0}", v)
renderer.material để đổi màu	sharedMaterial, hoặc lưu bản sao và Destroy
1 Canvas cho cả màn hình, text đổi mỗi frame	Canvas con cho phần động; gán khi giá trị đổi

Buổi Debug giúp mình **tìm ra** lỗi.

Hiểu chuyện gì xảy ra sau mỗi dòng code giúp mình **không viết ra** lỗi.

Hỏi đáp

Demo + tài liệu: github.com/vtcong2299/DEMO_UNITY · TalkDocs_UNITY/SauMoiDongCodeUnity.docx